

Advanced Programming

Property-Based Testing

Fritz Henglein

Datalogisk Institut, Københavns Universitet (DIKU)

- 1 Specification, implementation, verification
 - 2 Testing fundamentals
 - 3 Test design by input partitioning
 - 4 Randomized property-based testing
 - Example: specifying a state monad
 - 5 Making random testing effective
 - 6 Model-based testing of stateful systems
-

Goals

Week 4 (last week):

- free monads;
- separating the *use* of effects from their *interpretation*;
- programming against an abstract effect interface.

Week 5 (this week):

- specification, implementation, verification;
- specification-driven testing and input partitioning;
- randomised property-based testing with QuickCheck;
- generators, shrinking, and *measuring* test data quality;
- model-based testing of *stateful* systems.

Next week: concurrent programming – where testing gets genuinely hard.

Abstract data types and modules revisited

- An *abstract data type* (ADT) is an abstract type together with operations involving the type and a precise description of what the operations do. An ADT consists of
 - an *interface* containing the *declarations* of the type and its operations (functions),
 - their *specification* (what they do, which properties they have); and
 - *examples* illustrating correct and incorrect use of the operations.
- An *implementation* of an ADT contains
 - the *definitions* (source code) of the operations; and
 - *evidence* that it satisfies the ADT's specification (test design documentation and resulting test suite).
- An ADT typically has *multiple possible implementations*.
 - In particular, the implementation types may be different from each other.
 - Example: $s \rightarrow (a, s)$ and $\text{IORef } s \rightarrow \text{IO } a$ for state monads.
- *Module*: Structure with zero, one or more abstract or concrete type declarations.
 - ADT: Module with one abstract type declaration.

Abstract data types

- Programming language support for ADTs and *modular abstraction*:
 - ML-family languages: Signatures (module types), structures (module implementations), functors (module-parameterized structures).
 - Haskell: Type classes, class instances, class constraints in functions.
 - Java-like languages: Interfaces and abstract classes, concrete classes, concrete classes with interface-parameterized constructor(s).

Specifications

- *Specification*: Describes what a (collection of) function(s) does, not how it is implemented (its code).
 - Litmus test: Contains *everything* a user of a module *needs to know* and *nothing else*.
 - Specifications may be
 - inconsistent (contradictory): contain a contradiction;
 - incomplete (often the case): leave room for multiple implementations with different behaviours.
- *Implementation*: Code.
- *Correctness*: Evidence (e.g. test design method and resulting *test suite*) that code may satisfy its specification.
- *Requirements*: Informal description of desired functionality of a program component (what it should do and what it should not do).
 - Requirements may be unclear and leave room for interpretation.
 - A specification *resolves and documents* this ambiguity.

Module interface specification as a contract

- Module interface specification: *Contract* between programmer *using* a module (user) and programmer *implementing* it (implementor).
 - User to implementor: “All I want to know needs to be written in the module specification. How I use your implementation is none of your business. Don’t make me look at your code.”
 - Implementor to user: “All you need to know is written in the module specification. How I implement it is none of your business. Don’t ask to look at my code. (I may change it at any time.)”
 - Implementor and user are *roles*: They *may* be the same person.

When specification and implementation disagree

- What if specification and implementation are *inconsistent*, as evidenced by a *test failure*?
 - Change the code to meet the specification: Requires no negotiation with user(s).
 - Change the specification to meet the code: Requires negotiation with user(s) since *changes may propagate to whatever code they have written*.
 - Change both, code and specification: Natural part of *exploratory programming* during *requirements elicitation*.
 - Common problem: Most modules are *underspecified*. User tries out code and relies on unspecified behaviour of implementation. Implementor subsequently changes the code. Changes propagate to user code and cause hard-to-detect and hard-to-localise errors.

Testing

- *Testing* is (partial) verification by
 - executing code on inputs and checking whether the result is evidence of a specification violation;
 - *systematically* constructing the inputs to *maximise their likelihood of demonstrating a specification violation*.
- Terminology:
 - **Formal verification:** Mathematical proof that there exists *no* valid input that results in a specification violation.
 - **Testing:** The *systematic* discipline of finding “nasty” inputs that maximise the likelihood of finding a specification violation *quickly*.
 - Not being able to “break the code” is taken as evidence (not proof) that the code may be correct with respect to its given specification.
 - **Trying out, illustrating, exemplifying:** Applications of the code showing that it satisfies its specification for *some* inputs (if there is a specification) or that it does something useful (if there is no specification, only informal requirements).
- Don't use the term “test” unless you have *both* specification and code and are *systematically* looking for violations. Say “illustrate”, “exemplify” or “try out” instead.

Testing

- *Unit testing*: Testing a unit of code against its specification.
 - A *unit* typically corresponds to a module with a number of related types and functions.
 - Dependencies are *stubbed out*, so that an observed violation can be attributed to the unit itself, not to the code it depends on. A stub also flags calls that violate a dependency's *precondition*: that is an error in *your* unit even when nothing observable goes wrong.
- *Integration testing*: Testing a composition of modules against the specification of the resulting *component*. Most common: *bottom-up*, treating the component as one big “unit”.
- *System testing*: Testing a deployable system against its system specification – the whole system as one huge “unit” of code and data.
- *Acceptance testing*: Testing a *deployed* system in its target environment against business and user requirements, including usability and work flow.
- This lecture: property-based testing of small units, and model-based testing of larger (sub)systems.

Specification-driven testing

- Recall: All forms of testing require *both* specification and code.
 - No specification, no testing.
- *Specification-driven testing* is the systematic and documented *process* of analysing a specification to arrive at a *test suite with high falsification power*:
 - find *valid* data with maximal likelihood to expose a specification violation of the code;
 - determine corresponding *expected* outputs *from the specification*.
 - Don't look at the code in this process.
- Test-driven development (TDD):
 - Develop the test suite *before* you have any code.
 - Continuously run the test suite to drive and check the code as you develop it.
- Specification-driven testing is also called *specification-based testing*, *external testing*, and *black-box testing*.
 - “Black-box testing” is not used here since much of questionable value has unfortunately been written about it on the web.

Properties

- A *logical statement* is a statement that is either true or false.
- A *(logical) property* of one or more functions is a true logical statement involving the function(s).
- A *universally quantified property* has the form

$$\forall x_1, \dots, x_k. Q(x_1, \dots, x_k)$$

or

$$\forall x_1, \dots, x_k. (P(x_1, \dots, x_k) \Rightarrow Q(x_1, \dots, x_k)).$$

In words: For all (valid) $(x_1, \dots, x_k)$ the *postcondition* $Q(x_1, \dots, x_k)$ is true, where $(x_1, \dots, x_k)$ is valid if the *precondition* $P(x_1, \dots, x_k)$ is true.

- Example:

$$\forall x \in \text{Int} ((x \geq 0) \Rightarrow \text{fib}'(x) = \text{fib}(x))$$

where fib' is an optimised version of the Fibonacci function and fib is its canonical recursive definition, which serves as the *specification*, also called the *reference implementation*, for fib' .

Properties and specifications

- A *(partial) formal specification* is often a conjunction of universally quantified properties.
- *Property-based testing*: Systematic design of *finite* subset of inputs aimed at falsifying a property.
- In practice, a (full) specification typically consists of a partial formal specification and an informal (natural language) description.
- The rest of this lecture is about the two halves of that design problem:
 - which properties to state; and
 - which inputs to run them on.

Where do properties come from?

The hard part is not running the test cases. It is

- *property design*, that is, formulating *testable (executable) properties*, and
- *test design*, that is, methodical generation of test data that effectively and efficiently finds specification violations, if any exist.

Recurring patterns for property design:

- **Algebraic laws**: equational properties such as monad laws, associativity, commutativity, idempotence, neutral elements.
- **Invariants**: properties that must be preserved by operations, e.g.
`sorted xs ==> sorted (insertSorted x xs)`
- **Round-trip (inverse) properties**: operations that “undo” each other, e.g.
`parse (print e) = e,`
`decode (encode x) = x`

Where do properties come from? (continued)

- **Consistency with a reference implementation:** a simple, obviously correct, but maybe inefficient version is an *executable specification* – a *model*.
- **Metamorphic properties:** a property that relates multiple runs of the same function to each other; e.g.
`sort (xs ++ ys) = sort (ys ++ xs)`
- **Relational properties:** a property that relates *two or more* functions on the same or related inputs to each other; e.g.
`qsort xs = msort xs`. This is the essence of *model-based testing (MBT)*.
- **Direct properties**, also called *oracle-based* properties: a property that relates the output and the input of a function; e.g.
`sorted (msort xs)`

Note that a specification typically consists of a *conjunction* of multiple universally quantified properties. For example, `sorted (msort xs)` is satisfied by `const []`. A sorting function must also satisfy that its output is a permutation of its input.

Test design: Input partitioning

- *Specification-driven input partitioning* is a test design method for constructing *test data* by *inspecting the properties to be checked*.
 - Input partitioning is also called *equivalence partitioning* or *equivalence class partitioning*.
- Given a universally quantified property, partition its valid inputs into a *finite* set of partitions, that is into pairwise disjoint subsets whose union is the entire set of valid inputs.
- From each partition,
 - choose a *typical* element and one or more *boundary elements*;
 - determine the *expected* output for each element according to the function specification—*not* the code.
- Test suite: The derived test data and the property they are for. Partitioning is driven by finding boundaries between blocks of the partition that increase the likelihood of finding a specification violation.
- Test execution: Check the property by executing its implementation on the inputs in the test suite.

Input partitioning: Numbers

- Generally useful partitioning of integral and floating-point numbers.
 - `int` (32-bit 2's-complement integers):
 - Partitions: Negative numbers, `{0}`, positive numbers.
 - Values: `-2147483648`, `-137`, `-1`, `0`, `1`, `2377`, `2147483647`.
 - `float` (64-bit IEEE 754 floating-point numbers):
 - Partitions: `{ Negative infinity }`, negative numbers, `{0}`, positive numbers, `{ positive infinity }`, `{ NaN }` ("not-a-number").
 - Values: `NegativeInfinity`, `-1.7976931348623158e+308`, `-1.0`, `-Epsilon`, `0.0`, `Epsilon`, `1.0`, `1.7976931348623158e+308`, `PositiveInfinity`, `NaN`.
- Note that most test values are boundary values.
- Note also: `0.0 == -0.0` is true, but `NaN == NaN` is false.
Boundary values are where *assumptions* a programmer may have relied on break down.

Intermediate summary:

Programming = Specification + coding + verification

■ Programming consists of

- 1 *Specification*: What is computed? Which functions, what are their *properties*?
- 2 *Coding*: How is it computed? How are the functions implemented (“code”)?
- 3 *Verification*: Which explicit *evidence* is there that the program code meets its specification (correctness)?

■ Verification always requires *both*, program code *and* specification.

- Program code may be correct with respect to one specification and incorrect with respect to another specification.

■ Avoid statements such as “My code works”.

- With respect to which specification? Code is trivially correct with respect to whatever it happens to do—there is nothing to test; it does what it does. But which *properties* does it have that a user can rely on?
- Does it satisfy the specification for *all* valid data? Or just for *some* “friendly” data you happen to have thought of? Can you provide evidence that it may satisfy its specification by execution on “nasty” data that are *designed to break* the code?

Property-based randomised testing

- Consider an implementation of a module specification.
- Let the module specification contain a *partial formal specification* in the form of a *conjunction of universally quantified properties* that involve the module's functions.
 - The module specification may contain additional requirements.
- *Randomised property-based testing*: For each property S of the form $\forall x_1, \dots, x_k. (P(x_1, \dots, x_k) \Rightarrow Q(x_1, \dots, x_k))$
 - generate test data, *valid* tuples $(v_1, \dots, v_k)$ satisfying $P(x_1, \dots, x_k)$, (*pseudo*)*randomly* and evaluate $Q(v_1, \dots, v_k)$;
 - if an evaluation *fails* (returns *False*), this *proves* that S is *not satisfied* by the module implementation;
 - if all evaluations *succeed* (return *True*), this *provides empirical evidence* that the module implementation *may satisfy* S .
- Test design consists of systematically generating test data
 - with no or few invalid tuples and
 - a high *detection rate*, that is, a high proportion of the test data that manifest a specification violation compared to the proportion in all the valid data.

QuickCheck: Defining properties

```
quickCheck :: Testable a => a -> IO ()
```

```
class Testable a where
  property :: a -> Property
```

```
data Property
```

```
forall    :: Testable b => Gen a -> (a -> b) -> Property
(==>)     :: Testable a => Bool -> a -> Property
(==)      :: (Eq a, Show a) => a -> a -> Property
counterexample :: Testable a => String -> a -> Property
ioProperty      :: Testable a => IO a -> Property
withMaxSuccess  :: Testable a => Int -> a -> Property
```

- *Property*: probabilistic computation that succeeds or fails, with information about the failure.
- *Testable type*: type with a default way of turning a value into a property.
- Use `==` rather than `==`: it prints *both* values on failure.

QuickCheck: What the combinators do

Each combinator contributes one part of $\forall x_1, \dots, x_k. (P \Rightarrow Q)$.

- `forAll g f` is the *quantifier*: draw x from the generator g and test $f\ x$.
 - A property written as a *function* also quantifies, but always with arbitrary. `forAll` is where *test design* enters.
- `p ==> q` is the *premise* P : if p is **False** the test case is *discarded* – neither a success nor a failure – and another is drawn.
- `x === y` is a *conclusion* Q that is an equation: like `==`, but reports *both* values when they differ.

The remaining three change how a property is *reported* or *run*, not what it states:

- `counterexample s q`: if q fails, print s as well.
- `ioProperty a`: let Q be computed with *side effects*.
- `withMaxSuccess n q`: require n successful test cases, not the default 100.

QuickCheck: Pseudorandom generation of values

```
newtype Gen a = MkGen {unGen :: QCGen -> Int -> a}
```

```
class Arbitrary a where
```

```
  arbitrary :: Gen a
```

```
  shrink    :: a -> [a]
```

```
choose    :: Random a => (a, a) -> Gen a
```

```
elements  :: [a] -> Gen a
```

```
oneof     :: [Gen a] -> Gen a
```

```
frequency :: [(Int, Gen a)] -> Gen a
```

```
sized     :: (Int -> Gen a) -> Gen a
```

```
sample    :: Show a => Gen a -> IO ()
```

- `Gen a`: computation producing a (pseudo)random value of type `a`.
Seed threaded like a state monad; *size* available like a reader monad.
- `Arbitrary`: types with a default generator (and shrink function).
- Recommendation: Always run `sample` on a generator you have just written.

Example: Specifying a state monad

- In Haskell an ADT is declared by a *class*, an implementation by an *instance*. The properties belong to the class – but only as documentation.
 - Haskell checks an instance's *types* statically, not its laws.
 - Use property-based testing of the laws.
- Recall StateMonad from Chapter 4:

```
1 class (Monad m) => StateMonad m s | m -> s where
2   get  :: m s
3   put  :: s -> m ()
```

- Specification: the monad laws, plus three *state laws*

```
1 put s >> get      = put s >> pure s      -- put-get
2 put s >> put s'    = put s'              -- put-put
3 get >=> put        = pure ()              -- get-put
```

Two implementations of one interface

```
-- Chapter 2
```

```
newtype State s a = State (s -> (a, s))  
runState :: s -> State s a -> (a, s)
```

```
-- Chapter 4
```

```
newtype IState s a = IState {runIState :: IORef s -> IO a}
```

- Both are instances of StateMonad: correctly typed operations – but do they satisfy the laws?
- To observe an IState we define a run function that returns both result and state.

```
runIStateFull :: s -> IState s a -> IO (a, s)  
runIStateFull s c = do  
  ref <- newIORef s  
  a <- runIState c ref  
  s' <- readIORef ref  
  pure (a, s')
```

Observational equivalence

- The laws use equality, $=$, but what does it mean for expressions to be equal?
- *Observational equivalence*: Two expressions are equal if they are intersubstitutable in *every* program context without changing the *observable behaviour* of the program.
- Equality depends on the operations available in program contexts. Example: Monads.
 - Without a `run` function: All monad expressions are observationally equivalent.
 - With a `run` function: Different monad expressions may be observationally distinguishable.
 - Here: We have `run` functions `runState` and `runIStateFull`.

Testing observational equivalence

- Observational equivalence of State **Int** a expressions as a QuickCheck property:

```
(~=) :: (Eq a, Show a)
      => State Int a -> State Int a -> Property
c1 ~= c2 = property $ \s -> runState s c1 === runState s c2
```

- Simulation relation between State **Int** a and IState **Int** a as a QuickCheck property:

```
simulates :: (Eq a, Show a)
           => IState Int a -> State Int a -> Property
c 'simulates' r = property $ \s -> ioProperty $ do
  x <- runIStateFull s c
  pure (x === runState s r)
```

Monad laws as QuickCheck properties

```
prop_putGet s      = (put s >> get)      ~ = (put s >> pure s)
prop_putPut s s'   = (put s >> put s') ~ = put s'
prop_getPut        = (get >=> put)      ~ = pure ()

prop_simGet        = get      'simulates' get
prop_simPut s      = put s    'simulates' put s
prop_simPure x     = pure x   'simulates' pure x
```

- Each simulation property mentions its program *once*: `get`, `put` and `pure` are overloaded, so the same expression denotes a computation in `IState Int` on the left and in `State Int` on the right.
- Testing that an implementation simulates a reference is, here, just running *one* program under *two* instances of the same class.
- Caveat: finitely many initial states, and `s` fixed to `Int`. Sound but incomplete – observational equivalence is generally undecidable.

Generating functions

- Testing properties involving ($\gg=$) requires generating functions.

```
data Fun a b = ...  
applyFun :: Fun a b -> (a -> b)  
prop_simBind :: Fun Int Int -> Property  
prop_simBind fun =  
  (get  $\gg=$  put . f) 'simulates' (get  $\gg=$  put . f)  
where f = applyFun fun
```

- *Generation*: the *argument* perturbs the random seed, so each argument selects a different draw from b's generator. The result is a closure: opaque.
- *Printing*: Fun also carries a lazily built *table* of the argument–result pairs the test actually used, with a default:
 $\{x1 \rightarrow y1, x2 \rightarrow y2, _ \rightarrow d\}$.
- *Limitation*: this tests $\gg=$ only at continuations of shape `put . f`. Arbitrary continuations need random *computations* – that is model-based testing, below.

Input partitioning versus random testing

- Specification-driven and code-driven input partitioning: Good at identifying boundary elements/corner cases with high falsification potential.
- Random testing: Good at automatically generating large numbers of typical (random) elements.
- QuickCheck can be used to combine both: *custom generators* reflecting the outcome of input partitioning design.
- Conventional wisdom: Combine them.
 - Use type-based pseudorandom generators, without regard to specification or code.
 - Estimate their detection rate (quality).
 - Refine using specification-driven input partitioning.
 - Check the code coverage of the test suite.
 - Add test cases to improve code coverage.
- Challenge: generating *only* or *mostly* valid data.
 - Invalid data have *no* falsification power.

Generating valid data

```
prop_insertSorted :: Integer -> [Integer] -> Property
prop_insertSorted x xs =
  sorted xs ==> sorted (insertSorted x xs)
```

Looks fine. Now ask QuickCheck what it actually tested (classify, label, collect, tabulate):

```
*** Gave up! Passed only 66 tests; 1000 discarded tests (65% trivial).

length of xs (66 in total):
64% 1-4
36% 0
```

- Random lists are almost never sorted, so ==> discarded 1000 cases.
- The survivors were sorted *by accident*: length 0, 1 or 2.
- The property “passes” and has tested essentially nothing.
- **A rarely satisfied precondition does not merely waste time – it silently biases the test data.**

Fix 1: generate values that satisfy the precondition

```
genSorted :: Gen [Integer]
genSorted = sort <$> arbitrary

prop_insertSorted'' x =
  forAll genSorted $ \xs ->
    tabulate "length of xs" [lengthBucket xs] $
      sorted (insertSorted x xs)
```

+++ OK, passed 100 tests.

```
length of xs (100 in total):
46% >=20
31% 5-19
17% 1-4
6% 0
```

- No discards, healthy spread of lengths. Are we done?

Fix 2: state the partitioning as a coverage requirement

cover p b s asserts that at least p% of tests satisfy b;
 checkCoverage turns a shortfall into a *failure*.

```
prop_insertSortedCover x = forAll genSorted $ \xs ->
  checkCoverage
    . cover 2 (null xs)                                "empty list"
    . cover 20 (not (null xs) && x < head xs)           "insert at front"
    . cover 20 (not (null xs) && x > last xs)           "insert at back"
    . cover 20 (x `elem` xs)                             "insert duplicate"
  $ sorted (insertSorted x xs)
```

```
*** Failed! Insufficient coverage (after 400 tests):
20.8% insert duplicate
 7.5% insert at front
 7.0% insert at back
 6.0% empty list
Only 7.5% insert at front, but expected 20.0%
```

- x is generated *independently* of xs: in a long list it lands in the middle.

Fix 3: let the test design drive the generator

```
genInsertion :: Gen (Integer, [Integer])
genInsertion = do
  xs <- genSorted
  x <- frequency $ (2, arbitrary) : if null xs then [] else
    [ (3, pure $ head xs - 1)      -- before all elements
    , (3, pure $ last xs + 1)      -- after all elements
    , (3, elements xs) ]          -- equal to an existing element
  pure (x, xs)
```

```
+++ OK, passed 800 tests:
28.0% insert at back
28.0% insert duplicate
27.3% insert at front
5.2% empty list
```

- **The synthesis:** partitioning says what the distribution *ought* to be; cover makes that intention machine-checkable; the generator is written to satisfy it; randomness fills in typical values within each partition.

Shrinking

- A random counterexample is usually far bigger than necessary.
- `shrink :: a -> [a]` produces “slightly smaller” values. When a counterexample `x` is found, QuickCheck tests `shrink x`, and recurses on the first shrink that still fails.

```
instance (Arbitrary a, Arbitrary b)
=> Arbitrary (Pair a b) where
  arbitrary = Pair <$> arbitrary <*> arbitrary
  shrink (Pair x y) =
    [Pair x' y | x' <- shrink x]
    ++ [Pair x y' | y' <- shrink y]
```

- Shrink components *one at a time*: fewer candidates, and simultaneous shrinks are reached in two steps anyway.

Shrinking: two things to get right

- **Rule:** every value in `shrink x` must be *strictly smaller* than `x` under some well-founded ordering.
 - Otherwise shrinking loops forever – exactly when you have found a bug.
 - `shrink _ = []` is always legal, and a fine starting point.
- **Pitfall:** shrinking ignores preconditions.
 - A generator that maintains an invariant, plus a `shrink` that does not, gives you counterexamples that were never legal inputs.
 - With a custom generator use `forallShrink`, and make the `shrink` function preserve the invariant.

Running properties: Tasty

```
testProperty    :: Testable a => TestName -> a -> TestTree
testProperties  :: TestName -> [(String, Property)] -> TestTree
```

```
$ cabal test --test-option --quickcheck-tests=10000
$ cabal test --test-option --quickcheck-replay=42
$ cabal test --test-option --quickcheck-verbose
```

- Number of tests: trade time for confidence.
- *Replay*: fix the seed, make a failing run reproducible.
 - A property that fails intermittently in continuous integration and passes on your machine is a property whose seed you did not record.

The problem with state

- So far: pure functions, properties relating inputs to outputs.
- Most real systems are *stateful*: internal data modified by *commands*, partially revealed by *queries*.
 - A mutable data structure, a database, a network service, a robot controller.
- There is no “input” and “output” to relate – only a *history* of interactions.
- Test design: partition the *state space*. Design *commands* for reaching boundary and typical states. Use *queries* to observe the reached state.
- *Command–query separation* (CQS) as a design pattern for testability: every operation either changes state and returns nothing, or returns something and changes nothing.
 - If a stateful system is hard to model, the absence of an operation that observes the state without mutating it is often the reason.
- In industry this is arguably the *most impactful* use of property-based testing.

Model-based testing: the method

- *Software under test* (SUT): the implementation as it actually is, including all its operational detail.
- *Model*: a simple executable specification of the SUT's *observable* behaviour.
- *Commands and responses*: an abstract *application programming interface* (API) shared by both.
- *Program*: a sequence of commands.

```
step :: Model a -> Command a -> (Model a, Response a)  -- model
exec :: DynamicArray a -> Command a -> IO (Response a) -- SUT
```

- The SUT need not be written in Haskell, or even run on this machine!
 - Model in Haskell, SUT a network service, a C library via the *foreign function interface* (FFI), a robot.
 - Only exec has to change.

Model-based testing: the property

- For *every* program, the SUT and the model produce the *same responses*.

```
runProgram :: Eq a =>
  DynamicArray a -> Model a -> Program a -> IO Bool

prop_array :: Program Int -> Property
prop_array prog = ioProperty $ do
  c <- newDynamicArray
  runProgram c initModel prog
```

- runProgram issues each command to both and stops at the first disagreement.
- ioProperty is needed because the SUT runs in IO.
- The quantifier is *implicit*: prop_array is a function, so its argument is drawn from arbitrary :: Gen (Program Int).

Example: a dynamic (resizable) array

```

data DynamicArray a = DynamicArray      -- SUT: index fiddling
  { daUsed :: IORef Int, daCap :: IORef Int
  , daArr  :: IORef (IOArray Int a) }
newtype Model a = Model [a]            -- model: a sequence
data Command a = Insert a | Index Int
                  | Write Int a | Delete Int
data Response a = Success | Failure | Elem a

```

- Initial capacity 10; double on overflow; halve when less than half full. Semantically it just maintains a sequence – all the complexity is *operational*.
- The model is 20 lines; the SUT is 80. The gap only grows with real systems.

The naive generator

```
instance Arbitrary a => Arbitrary (Command a) where
  arbitrary = oneof
    [ Insert <$> arbitrary, Index <$> arbitrary
    , Write <$> arbitrary <*> arbitrary
    , Delete <$> arbitrary ]
```

```
> sample (arbitrary :: Gen (Program Int))
Program []
Program []
Program [Insert (-4),Index (-3),Index 0]
Program [Delete 1,Delete 2,Delete (-4),Write 4 4]
Program [Index (-5),Delete 1,Index 6,Write 7 (-8),Index (-4)]
```

- quickCheck prop_array says
+++ OK, passed 100 tests..
- Almost every command is out of bounds; the array rarely holds anything.
- Is this a good test suite? *Measure it.*

Partitioning the state space

```
programCoverage prog = checkCoverage      -- from the model trace
  . cover 30 (maxElems prog > 10)         "grows past capacity"
  . cover 10 (deletedWhenLarge prog)      "deletes while large"
  . cover 30 (anyRejected prog)           "command out of bounds"
```

```
*** Failed! Insufficient coverage (after 800 tests):
```

```
94.5% command out of bounds
```

```
18.9% grows past capacity
```

```
16.5% deletes while large
```

```
...
```

```
Only 18.9% grows past capacity, but expected 30.0%
```

- The interesting states are around the capacity boundaries.
- Fewer than one program in five ever resizes the array: the resizing code – by far the most intricate part – is barely tested at all.

Generating command sequences that make sense

```

genCommand :: Arbitrary a => Model a -> Gen (Command a)
genCommand (Model xs) = frequency $ unconstrained ++ inBounds
  where
    unconstrained =
      [ (insertWeight, Insert <$> arbitrary)
      , (1, Index <$> arbitrary)
      , (1, Write <$> arbitrary <*> arbitrary)
      , (1, Delete <$> arbitrary) ]
    inBounds | null xs    = []
              | otherwise =
                [ (3, Index <$> validIndex)
                , (3, Write <$> validIndex <*> arbitrary)
                , (3, Delete <$> validIndex) ]
    validIndex  = chooseInt (0, length xs - 1)
    insertWeight = max 2 (30 - 2 * length xs)

```

- The weight of Insert depends on the *current state*, so programs drive the array to an interesting size and stay there. Generating a program is a *fold over model states*.

Does it matter? Reintroduce a real bug

Delete one line from `insert`, so that a `resize` loses the element that triggered it. Then run each version 20 times at the default 100 tests:

generator	coverage met?	runs that caught the bug
naive	no	7 out of 20
state-aware	yes	20 out of 20

- A test suite that reports success two times out of three on definitely broken code is *worse than no test data at all*: it gives the impression of being a test suite – which is designed to maximise the likelihood of finding an error – when it is nothing of the sort.
- The coverage check tells you – *before* you rely on it – that your test data never reaches the code you care about.
- For stateful systems the hard part is almost never the model. It is generating command sequences that reach interesting states. Plan your effort accordingly.

Practical matters

- **Make failures informative:** `runProgram` returning **Bool** tells you *that* a program failed, not *which command* diverged. Use `counterexample` to report the command, the model state and both responses. This costs four lines and saves a great deal of debugging.
- **Preconditions:** if a command must not be issued in some state, both *generation* and *shrinking* must respect that – dropping an earlier command can invalidate a later one.

```
1 shrinkProgram (Program cmds) = filter validProgram
2   $ map Program $ shrinkList (const []) cmds
```

- **Libraries:** `quickcheck-state-machine`, `quickcheck-dynamic` package all of this. We build it by hand because you will need to adapt it.
- **Cost:** our test harness is longer than the SUT. Fair objection – but real systems grow much faster than their models.

Testing challenges

■ Testing concurrent code:

- Properties need to hold *for all possible schedulers* (all interleavings of all concurrently executing threads), not only for the particular scheduler one happens to use.
- The usual specification is *linearisability*: the observed concurrent behaviour must be explainable by *some* sequential ordering respecting the real-time order of non-overlapping operations.
- Requires exploring many schedules *and* searching for a valid linearisation. Very difficult *and* expensive; beyond the scope of Advanced Programming – but built on exactly the model-based foundation above.

Further sources of error

- Confusing test design with test suite:
 - The test suite is the *output of the test design process*. By itself it says nothing about its falsification power. Only the *test design report* documenting the *systematic process* of constructing the test suite does.
- Nondeterminism: with `ioProperty` the outcome can depend on more than the generated data – on state left behind by earlier test cases, on timing, on the environment.
 - Shrinking re-runs the property on candidate values and keeps those that still fail, so it is then searching on noise: it stops early, or reports a counterexample that does not fail when run on its own. Create the system under test *inside* the property.
 - Note that this is *not* the same as a property failing on some generated data and not on other data: that is what random testing is *for*.
- Draw all test data *before* `ioProperty` – from the property's arguments or with `forAll` – and let the **IO** action only perform effects.
 - Values drawn inside it with `generate` are invisible to QuickCheck: not reported, not shrunk, not replayable.

QuickCheck: More information

- Course notes, Chapter 5: https://diku-dk.github.io/ap-notes/chapter_5.html
- QuickCheck library documentation: <https://hackage.haskell.org/package/QuickCheck/docs/Test-QuickCheck.html>
- QuickCheck manual: <https://www.cse.chalmers.se/~rjmh/QuickCheck/manual.html>
- Claessen & Hughes, *QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs*, ICFP 2000.
- Hughes, *How to specify it! A guide to writing properties of pure functions*, TFP 2019. (Strongly recommended – it is essentially the “where do properties come from” slide, at length.)

Summary

- 1 Specification, implementation, verification
- 2 Testing fundamentals
- 3 Test design by input partitioning
- 4 Randomized property-based testing
 - Example: specifying a state monad
- 5 Making random testing effective
- 6 Model-based testing of stateful systems